

1. CREACIÓN Y SELECCIÓN DE LA BASE DE DATOS

```
CREATE DATABASE IF NOT EXISTS ferrucho;
```

```
USE ferrucho;
```

Explicación:

```
CREATE DATABASE IF NOT EXISTS ferrucho;
```

Crea una nueva base de datos llamada ferrucho si no existe ya.

IF NOT EXISTS evita un error si la base de datos ya está creada.

El nombre ferrucho es el identificador de la base de datos donde se almacenarán todas las tablas.

```
USE ferrucho;
```

Selecciona la base de datos ferrucho como la base de datos activa para todas las operaciones siguientes. Sin esta línea, las tablas se crearían en la base de datos por defecto (o se produciría un error).

2. CREACIÓN DE LA TABLA CLIENTES

sql

```
CREATE TABLE clientes (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    nombre VARCHAR(50) NOT NULL,  
    ciudad VARCHAR(40),  
    telefono VARCHAR(15),  
    email VARCHAR(60)  
);
```

Explicación:

```
CREATE TABLE clientes ( ... );
```

Define una nueva tabla llamada clientes.

Columnas:

```
id INT PRIMARY KEY AUTO_INCREMENT:
```

INT: tipo de dato entero.

PRIMARY KEY: establece que esta columna es la clave primaria, es decir, identifica de forma única cada fila.

AUTO_INCREMENT: hace que el valor se genere automáticamente (1, 2, 3, ...) al insertar un nuevo registro, sin necesidad de proporcionarlo manualmente.

nombre VARCHAR(50) NOT NULL:

VARCHAR(50): cadena de texto de hasta 50 caracteres.

NOT NULL: obliga a que este campo siempre tenga un valor (no puede estar vacío).

ciudad VARCHAR(40): cadena de hasta 40 caracteres, puede ser NULL (opcional).

telefono VARCHAR(15): número de teléfono como texto (hasta 15 caracteres), opcional.

email VARCHAR(60): correo electrónico, opcional.

Propósito:

Almacenar información básica de los clientes de la ferretería (o negocio). La clave primaria id permite referenciar a cada cliente de manera única.

3. CREACIÓN DE LA TABLA PRODUCTOS

sql

```
CREATE TABLE productos (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    nombre VARCHAR(60) NOT NULL,  
    precio DECIMAL(10,2) NOT NULL,  
    stock INT NOT NULL,  
    categoria VARCHAR(40)  
);
```

Explicación:

```
CREATE TABLE productos ( ... );
```

Define una tabla llamada productos.

Columnas:

id INT PRIMARY KEY AUTO_INCREMENT: similar a la tabla anterior, clave primaria autoincremental.

nombre VARCHAR(60) NOT NULL: nombre del producto, obligatorio.

precio DECIMAL(10,2) NOT NULL:

DECIMAL(10,2): número decimal con hasta 10 dígitos en total, de los cuales 2 son decimales. Por ejemplo, 12500.00 se almacena exactamente con dos decimales.

NOT NULL: el precio es obligatorio.

stock INT NOT NULL: cantidad disponible en inventario, entero y obligatorio.

categoría VARCHAR(40): clasificación del producto (ej. "Herramientas manuales"), opcional.

Propósito:

Gestionar el inventario de productos, con su precio y cantidad en stock.

4. INSERCIÓN DE DATOS EN CLIENTES

sql

```
INSERT INTO clientes (nombre, ciudad, telefono, email) VALUES
('Juan Pérez', 'Buenos Aires', '11-2345-6789', 'juanp@email.com'),
('María Gómez', 'Córdoba', '351-9876-5432', 'maria.gomez@mail.com'),
('Carlos Ruiz', 'Rosario', '341-4567-8901', 'carlos.ruiz@ferreteria.com'),
('Laura Fernández', 'Mendoza', '261-1234-5678', 'laura.fernandez@proveedor.com'),
('Diego Torres', 'La Plata', '221-8765-4321', 'diego.torres@negocio.com');
```

Explicación:

INSERT INTO clientes (nombre, ciudad, telefono, email)

Especifica que vamos a insertar filas en la tabla clientes, indicando explícitamente las columnas que se van a poblar (no incluimos id porque se genera automáticamente).

VALUES (...), (...), ...

Cada paréntesis contiene los valores para una fila, en el mismo orden que las columnas listadas.

Se insertan 5 registros con nombres, ciudades, teléfonos y correos.

El campo id se asignará automáticamente (1, 2, 3, 4, 5).

Propósito:

Cargar datos de ejemplo de clientes para poder realizar consultas y pruebas.

5. INSERCIÓN DE DATOS EN PRODUCTOS

sql

```
INSERT INTO productos (nombre, precio, stock, categoria) VALUES
('Martillo de carpintero', 850.50, 20, 'Herramientas manuales'),
('Taladro eléctrico 600W', 12500.00, 5, 'Herramientas eléctricas'),
('Cinta métrica 5m', 320.00, 35, 'Medición'),
('Llave ajustable 12"', 1100.75, 12, 'Herramientas manuales'),
('Lija para madera N°80', 180.25, 50, 'Insumos');
```

Explicación:

INSERT INTO productos (nombre, precio, stock, categoria)

Inserta filas en la tabla productos especificando las columnas.

Valores: se proporcionan el nombre, precio, stock y categoría de 5 productos distintos.

El id se genera automáticamente.

Los precios tienen dos decimales, respetando el tipo DECIMAL(10,2).

El stock es un número entero.

Propósito:

Poblar la tabla de productos con datos de muestra para probar consultas.

6. CONSULTAS DE VERIFICACIÓN

sql

```
SELECT * FROM clientes;
```

```
SELECT * FROM productos;
```

Explicación:

```
SELECT * FROM clientes;
```

Recupera todas las columnas (*) de la tabla clientes y las muestra. Es útil para comprobar que los datos se insertaron correctamente.

```
SELECT * FROM productos;
```

Igual, pero para la tabla productos.

Propósito:

Verificar visualmente que las inserciones se realizaron con éxito y que los datos son los esperados.

Resumen de buenas prácticas y observaciones

Claves primarias: ambas tablas usan id como clave primaria con AUTO_INCREMENT, lo que simplifica la inserción y garantiza la unicidad.

Tipos de datos adecuados: se usan VARCHAR para textos cortos, DECIMAL para precios (evita errores de redondeo con FLOAT) e INT para cantidades enteras.

Restricciones NOT NULL: se aplican a campos que siempre deben tener información (nombre, precio, stock), lo que evita datos incompletos.

Inserciones masivas: con una sola sentencia INSERT se cargan varios registros, lo que es más eficiente que hacer inserciones individuales.

Consultas de verificación: siempre es recomendable ejecutar SELECT después de las inserciones para confirmar que los datos se almacenaron correctamente.